

2025학년도 1학기
자연어처리 프로젝트
기말 보고서

대법원 판결 요지 자동 요약기
- 법률 비전문가를 위한 핵심 판결 분석 도구



과목명	자연어처리프로젝트
담당	구영은 교수님
제출일	2025년 6월 21일
문과대학	영어영문학과
2학년	2023310258
성명	허서정

목차

I. 서론

1. 기획 의도 및 필요성
2. 문제 제기와 목표 설정

II. 서비스 개요 및 처리 흐름

1. 서비스 개요: '판결문 요지 자동 요약기'
2. 주요 기능 및 사용자 대상
3. 처리 단계 요약

III. 데이터 수집 및 구조 설명

1. 수집 대상
2. 데이터 출처 및 수집 방식
3. 판결문 구조 설명
4. 수집 데이터 일부 예시

IV. 데이터 전처리 (Data Preprocessing)

1. 환경 세팅 및 라이브러리 설치
2. (서브 시스템 1) 전체 데이터의 판결별 결과 출력 - 상고 기각 vs 원심판결 파기
3. (서브 시스템 2) 전체 데이터의 판례명만 출력
4. 데이터 전처리 - Tokenization, Stopword Removal (Step 1)

V. 언어학적 분석 방법론 및 메인 서비스 구현

1. 한글 형태소 분석기 선정 과정
2. POS Tagging 및 사건별 고유 동사 출력 (Step 2)
3. PMI 기반 Bigram + Trigram Collocation 분석 (Step 3)
4. 사건별 중요 문장 추출 (TF-IDF 기반) (Step 4)
5. 불필요한 표현 제거 및 문장 정제 (Step 5)
6. 쉬운 말 해설 추가 (용어 설명 사전 적용) (Step 6)

VI. 최종 결과 출력물

VII. 결론 및 한계

1. 초기 목표와 달라진 부분
2. 서비스 성과 요약
3. 한계점
4. 개선 방안, 향후 연구 활용 방안

부록) 소감

I. 서론

1. 기획 의도 및 필요성

법은 사회 전반을 규율하며 시민들의 삶에 실질적인 영향을 미치지만, 많은 사람들이 법률 용어나 판결문의 복잡한 구조 때문에 법적 판단의 요지나 의미를 쉽게 이해하지 못하는 현실이 존재한다. 물론 이러한 문제를 개선하려는 시도도 있었지만, 여전히 대부분의 판결문은 전문적인 법률 용어와 문체로 작성되어 있어 일반 국민이 접근하고 이해하기에는 어려움이 크다. 판결문의 핵심 내용을 파악하는 데 시간이 오래 걸리고, 판결의 결과나 그 이유가 명확히 드러나지 않는 경우도 많아, 일반 시민들이 실생활에 활용하기 어렵다는 한계가 존재한다.

2025년 상반기에는 헌법재판과 대선을 계기로 형법과 헌법에 대한 범국민적 관심이 높아졌고, 자연히 대법원 판결에도 이목이 집중되는 흐름을 확인할 수 있었다. 이러한 상황 속에서 25년 상반기 대법원 홍보대사 역할을 수행하는 대법원 영블로거위원회 및 영상팀으로 활동하며 대법원 판결을 일반 국민이 이해하기 쉬운 형태로 가공, 전달하는 업무를 직접 수행하였다. 현재도 해당 업무를 지속하며, 그 과정에서 법률 정보의 접근성과 전달 방식에 있어 개선의 여지가 크다는 점을 절감하고 있다.

이러한 사회적 필요를 파이썬이라는 프로그래밍 언어와 자연어처리라는 방법론을 결합시켜 법률 정보의 접근성 불균형 문제를 해소해보고자 한다. 판결문을 텍스트 형식으로 수집하고, 이를 자연어처리를 통해 정보 수용자에게 이해하기 쉬운 형태로 가공하여 제공하는 서비스 개발을 통해 법률 정보의 보편화에 일부 기여하고 싶다.

2. 문제 제기와 목표 설정

이 프로젝트의 목적은 대법원 판결 요지로부터 핵심 판단 요소(행위 주체, 행위, 결론 등)를 자동으로 추출하고 요약하는 서비스를 개발하여, 법률 비전문가도 판결문의 주요 내용을 직관적으로 파악할 수 있도록 돕는 것이다. '원고'와 '피고'와 같은 법률 용어를 각각 '소송을 제기한 측', '소송을 당한 측' 등 보다 쉬운 표현으로 자동 변환하고, '원심 판단을 유지한다', '파기한다' 등의 표현을 분석해 대법원의 판단이 하급심과 일치하는지 시각적으로 명시할 예정이다. 또한 '배상하라', '무죄다', '파기환송한다'와 같은 결론을 요약해 제공하고, 자주 등장하는 핵심 단어는 그 의미를 바로 확인할 수 있도록 구성한다. 판결문에서 반복되는 동사 패턴도 분석해, 법률 문체의 구조적 특징을 드러내고 핵심 문장을 직관적으로 파악할 수 있게 할 계획이다.

II. 서비스 개요 및 처리 흐름

1. 서비스 개요: '판결문 요지 자동 요약기'

본 프로젝트는 i) 대법원 판결문에 포함된 핵심 판단 내용을 자동으로 추출하고, ii) 이를 일반 시민이 이해하기 쉬운 용어로 변환하여 iii) 간결하게 요약하는 자연어처리 기반 도구를 제작하는 것이다. 법률 비전문가도 판결의 요지를 빠르게 파악할 수 있도록 지원하는 것을 목적으로 한다.

2. 주요 기능 및 사용자 대상

a. 주요 기능은 하단 내용과 같다.

- 판결문에서 판단 동사를 중심으로 핵심 판단 문장 자동 추출 및 필터링 / 주어-동사-판단내용 구조 자동 추출 / 법률 용어에 대한 쉬운말 변환 / 핵심 요지 자동 요약 및 시각화 출력

b. 주요 사용자는 다음과 같다.

- 판결문을 쉽게 이해하고자 하는 일반 국민 / 언론·교육기관 종사자 / 판결문을 요약·해설 자료로 활용하려는 사용자 / 법학도 및 초기 법률 학습자 혹은 판결문의 구조적 이해에 도움을 받고자 하는 모든 사람

3. 처리 단계 요약

데이터 수집 → 전처리 → 분석 → 요약 및 시각화

- ① 데이터 수집: 대법원 홈페이지에서 최신 주요 판결문 수집
- ② 전처리: 문장 분할, 품사 태깅, 불용어 제거 등
- ③ 분석: 판단 동사 기반 문장 필터링, 구조 추출, 쉬운말 변환
- ④ 요약 및 시각화: 핵심 판단내용 자동 요약 및 리스트/시각화 형태 출력

Ⅲ. 데이터 수집 및 구조 설명

1. 수집 대상

- 본 프로젝트의 수집 대상은 대한민국 대법원이 선고한 주요 판결문이며, 그 중에서도 판결 요지와 판단 이유가 명확히 드러나는 민사 및 형사 분야 판례를 중심으로 선정하였다. 주요 주제는 계약 해석, 의무 불이행, 담보책임, 주거권 분쟁 등 실생활과 밀접한 사건 위주로 선정하였다.

2. 데이터 출처 및 수집 방식

- 본 프로젝트는 대법원 종합법률정보 포털(<https://www.scourt.go.kr/>)에 게시된 공개 판결문 HTML 페이지를 대상으로 데이터를 수집하였다.

- 초기에는 Python의 requests와 BeautifulSoup을 이용한 정적 크롤링을 시도하였으나, 해당 사이트는 동적 로딩 방식을 사용하고 있어 단순 HTML 파싱으로는 본문 접근이 불가능하였다. 이후 selenium을 활용한 동적 크롤링을 시도했으나, 페이지 이동 시에도 상단 URL 넘버링이 변화하지 않고, 요지(판결 요약)는 별도 팝업 형태로 렌더링되는 구조였기 때문에 자동화된 방식으로는 크롤링에 한계가 있었다.

- 이에 따라 실제 사용 가능한 판결문을 직접 수기로 수동 크롤링하여 전처리 과정을 거쳤으며, 최종적으로 총 10건의 판결문 텍스트를 lawtext.txt 형식으로 확보하여 분석에 활용하였다.

3. 판결문 구조 설명

- 대법원 판결문은 다음과 같은 논리적 구조를 가진다.

판결 제목 및 사건명

판결 요지 (요약): 사건의 핵심 판단 내용을 압축해 제시함

이유: 법리가 적용되는 근거, 사실 관계 및 법률 해석

판결 주문: 실제 판결 결과 (기각, 인용 등)

- 이 중 “판결 요지”와 “이유” 항목을 본 프로젝트에서 핵심 정보를 추출하는 주요 분석 대상으로 삼는다.

4. 수집 데이터 일부 예시

- <대법원 2023. 12. 7. 선고 2023다269139 판결> [소유권이전등기]

<아파트 매매계약서에 인도일과 실제 명도일 약정이 별도로 있는 경우 매도인의 현실인도의무 인정 여부가 문제된 사건>

【판결요지】

[1] 일반적으로 계약을 해석할 때에는 형식적인 문구에만 얽매어서는 안 되고 쌍방당사자의 진정한 의사를 탐구하여야 한다. ...

【이유】

원심은 이 사건 매매계약의 해석상 피고가 ... 원고의 잔금 지급의무 불이행을 원인으로 한 피고의 계약 해제 항변이 이유 있다고 판단하였다. ...

※ 실제 분석에 사용된 데이터는 txt 형태로 저장·전처리되며, 문단 구조와 핵심 판단문 위주로 구문 분리된다.

IV. 데이터 전처리 (Data Preprocessing) 및 원본 데이터 활용 서브 시스템 1, 2

1. 환경 세팅 및 라이브러리 설치

- NLTK와 KoNLPY를 설치한다.

KoNLPY 설치! (기본세팅 1)	NLTK 설치! (기본세팅 2)
<pre>[1] # 라이브러리 설치하기 !apt-get update !apt-get install g++ openjdk-8-jdk python-dev python3-dev !pip3 install JPype1-py3 !pip3 install konlpy # JAVA_HOME 환경변수 설정하기 %env JAVA_HOME "/usr/lib/jvm/java-8-openjdk-amd64"</pre>	<pre>#전부설치 import nltk nltk.download('all')</pre>

2. (서브 시스템 1) 전체 데이터의 판결별 결과 출력 - 상고 기각 vs 원심판결 파기

II - i. 상고 기각 vs 원심판결 파기	
<pre>import re # 1. 텍스트 불러오기 f = open('lawtext.txt', 'r') text = f.read() f.close() # 2. 판결문을 나누기 : <대법원 ... 판결> 이라는 글자를 기준으로 잘라서 리스트로 만들기 split_sent = re.split(r'(<대법원.*판결>)', text) # 이렇게 하면 사건당 제목, 내용 순으로 저장됨 # 3. 제목과 내용을 쌍으로 묶어서 저장하기 caselist = [] num = 1 while num < len(split_sent): title = split_sent[num] # 홀수 번호는 제목 바로 뒤의 짝수 번호는 내용 if num + 1 < len(split_sent): content = split_sent[num + 1] else: content = '' eachcase = [title, content] caselist.append(eachcase) num = num + 2</pre>	<pre># 4. 조건에 따라 분류 A_cases = [] B_cases = [] for case in caselist: title = case[0] body = case[1] body_lower = body.lower() if '원심판결' in body and '파기' in body: A_cases.append(title) elif '상고' in body and '기각' in body: B_cases.append(title) # 5. 출력 print(" A 유형 (원심판결, 파기):") for h in A_cases: print(h) print("\n B 유형 (상고, 기각):") for h in B_cases: print(h) print (''' A. 원심판결 파기 설명: '원심판결 파기' 는 대법원이 하급심(보통 고등법원 즉, 하급심 판결이 틀렸다고 본 것입니다. 파기되면 해당 예시 상황: 계약의 해석을 잘못한 경우 / 증거 채택이나</pre>

이 코드는 수동으로 수집한 대법원 판결문 텍스트(lawtext.txt)를 불러와 <대법원 20XX.XX.XX 판결> 패턴을 기준으로 사건별로 나눈 뒤, 제목과 본문을 쌍으로 묶어 리스트로 저장하고, 각 사건의 본문에 포함된 핵심 키워드를 기반으로 판결 결과를 자동 분류하도록 작성했다. 구체적으로는 본문에 '원심판결'과 '파기'가 함께 포함된 경우를 A 유형(원심 파기), '상고'와 '기각'이 함께 포함된 경우를 B 유형(상고 기각)으로 분류하며, 이를 통해 각 판결이 하급심 판결을 파기했는지, 그대로 확정했는지를 쉽게 파악할 수 있다. 이 결과는 향후 법리 분석이나 판결 경향 파악, 시각화 등 후속 작업을 위한 기초 자료로 활용될 여지가 있다.

특히 결과를 분류하여 A유형에 해당하는 판례를 쭉 나열하고, B유형에 해당하는 판례를 쭉 나열하

는 것을 통해 추후 대법원 판결 열람 사이트의 자동 크롤링이 가능하게 되면 대법원 판결에 대한 통계를 계산하는 것이 가능해질 것으로 예상된다.

3. (서브 시스템 2) 전체 데이터의 판례명만 출력

0. 제목 추출

```
import re

# 1. 파일 열어서 읽기
f = open('lawtext.txt', 'r')
text = f.read()
f.close()

# 2. < > 안에 있는 텍스트 찾기 (판결 제목)
title_list = re.findall(r'<([><]*)>', text)

# 3. 중복 제거 + 가나다순 정렬
final_titles = list(set(title_list))
final_titles.sort()

# 4. 출력
print('▶ 판결 제목 추출 결과:\n')

for i in range(len(final_titles)):
    print(f'({i+1}). {final_titles[i]}')
```

▶ 판결 제목 추출 결과:
1. 대법원 2023. 12. 7. 선고 2023C246600 판결
2. 대법원 2023. 12. 7. 선고 2023C269139 판결

이 코드는 수기로 데이터 수집하여 제작한 lawtext.txt파일에서 < > 안에 포함된 판결 제목들을 추출한 뒤, 중복을 제거하고 가나다순으로 정렬하여 번호를 붙여 출력하는 프로그램이다. 이를 통해 판결문 내에 등장하는 다양한 사건 제목들을 한눈에 확인할 수 있다.

4. 데이터 전처리 - Tokenization, Stopword Removal (Step 1)

```
[ ] # STEP 1 - 데이터 전처리 (문장 단위 토큰화 + 기호 위주 불용어 제거)
print('-----')
print('>>>>>> STEP 1. 데이터 전처리 (문장 단위 토큰화 + 기호 위주 불용어 제거) <<<<<<<')

import re
from konlpy.tag import Komoran

# 1. 텍스트 파일 열기
f = open('lawtext.txt', 'r')
text = f.read()
f.close()

# 2. 판결 제목 단위로 텍스트 나누기
blocks = re.split(r'(<대법원\s.*?판결>)', text)

# 3. 제목 + 본문 뒤에 사건별 리스트 생성
cases = []
for i in range(1, len(blocks), 2): # 홀수 index: 제목, 짝수 index: 본문
    title = blocks[i].strip()
    body = blocks[i+1].strip() if i+1 < len(blocks) else ''
    full_case = f'{title}\n{body}'
    cases.append(full_case)

# 4. 전처리 함수 정의
def preprocess_sentence(sentence):
    sentence = re.sub(r'[~!@#%&*~\^&^a-zA-Zws]', ' ', sentence)
    sentence = re.sub(r'([가-하])\b', ' ', sentence)
    sentence = re.sub(r'ws+', ' ', sentence)
    return sentence.strip()

# 5. 사건별 전처리 및 문장 단위 토큰화
preprocessed_cases = []
print('\n==== 전처리된 결과 ====')

for case_num, case in enumerate(cases, start=1):
    print(f'\n===== 사건 {case_num} =====')

# 6. 문장 단위 분리 (종결 표현 '다.', '요.' 기준)
sentences = re.split(r'(?<=[다요])\.\s*', case)

cleaned_sentences = []
for idx, sentence in enumerate(sentences, start=1):
    clean = preprocess_sentence(sentence)
    if clean:
        print(f'{idx}. {clean}')
        cleaned_sentences.append(clean)

preprocessed_cases.append(cleaned_sentences)
```

이 코드는 lawtext.txt파일에 저장된 대법원 판결문 데이터를 불러와, <대법원 ... 판결>형식을 기준으로 각 사건 단위로 분할한 후, 문장 단위로 나누고 불필요한 문장 기호를 제거해 전처리된 텍스트로 정리하는 과정을 포함한다. 추후 후처리 단계에서 전처리를 한 번 더 거치게 되는데, 여기서의 전처리는 원본 raw-text를 데이터 분석을 가능하게 하는 정도로 처리하는 가벼운 전처리를 의미한다. 문장 분리는 주로 '다.', '요.'로 끝나는 문장을 기준으로 했고, 이는 실제 판결문에서 자연스럽게 문장 경계를 구분하는 데 효과적이라고 판단해 이와 같이 진행하였다. 문장 분리 기준은 텍스트를 눈으로 먼저 보면서 기호들을 수집했고, 여러 테스트를 거쳐 (?<=[다요]w.)가 가장 적절하다고 판단하였다. 판결문 특성상 반복되는 표현이나 불필요한 문장기호 제거가 필요하다고 느껴 직접 정규표현식을 조정해가며 적용했고, 판결문 2개만을 저장한 텍스트 파일로 테스트하였을 때 문제가 없어 확장하여 적용하였다. 전체적으로 분석 준비를 위한 구조화된 형태로 데이터를 정리하는 데 초점을 두었다.

V. 언어학적 분석 방법론

1. 한글 형태소 분석기 선정 과정

언어학적 분석 step 1-1: 어떤 한국어 형태소 분석기 사용할 지 선택

```
from konlpy.tag import Komoran, Kkma, Okt, Hannanum

# 분석할 예제 문장
text = "대법원은 이를 공평과 신의칙에 반하는 것으로 판단하였다."

# 분석기 객체 생성
komoran = Komoran()
kkma = Kkma()
okt = Okt()
hannanum = Hannanum()

# 형태소 분석 실행
komoran_result = komoran.pos(text)
kkma_result = kkma.pos(text)
okt_result = okt.pos(text)
hannanum_result = hannanum.pos(text)

# 결과 출력
print("===== Komoran 분석 결과 =====")
for word, tag in komoran_result:
    print(f"{word}/{tag}", end=' ')
print("\n")

print("===== Okt 분석 결과 =====")
for word, tag in okt_result:
    print(f"{word}/{tag}", end=' ')
print("\n")

print("===== Hannanum 분석 결과 =====")
for word, tag in hannanum_result:
    print(f"{word}/{tag}", end=' ')
print()
```

===== Komoran 분석 결과 =====
대법원/NNP 은/JX 이/NP 를/JKO 공평/NNNG 과/JC 신의/NNNP 칙/

수업 시간에 들은 바에 따라, 한글 텍스트는 기본적으로 형태소를 단위로 분석한다고 하여, 한글 형태소 분석기를 활용하였다. 형태소는 뜻을 가진 가장 작은 단위이고, 한국어는 영어와 다르게 Topic - Comment Language이기에 특히 그 내용과 뜻을 확인하는 것이 중요하다. 아울러, 조사 어미 접사 등의 형식 형태소 또한 그 의미를 무시할 수 없고 의미를 성립하는 데 중요한 축을 담당하므로 형태소로의 분석은 타당하다.

다만 여기서 고민했던 지점은, 수업 시간에 한글 형태소 분석기 부분의 수업에서 들었던 바에 의하면 각 형태소 분석기는 분석 시 그 단위를 크고 넓게 보아 전체적 구조를 판단하는 데 중점을 두는지, 그 단위를 가능한 한 작게 잘라 세부적인 사항을 판단하는 데 중점을 두는지에 따라 선택해서 사용하면 된다는 지점이었다. 수업 시간 실습한 예제에서는 단순히 넷을 번갈아가며 사용해보는데 그치고 넷을 한 번에 활용하여 출력해 본 경험이 없었기에, 넷을 한 번에 활용하여 출력하는 실험을 설계하여 최종 Komoran 형태소 분석기를 선택하게 되었다.

Komoran을 선택한 이유는 본 프로젝트가 다루는 판결문 텍스트의 문어체적 특성과 법률 용어의 정확한 분해 필요성을 고려한 결과이다. Komoran은 다른 형태소 분석기(Okt, Hannanum, Kkma)에 비해 복합명사 분할 능력과 분석 일관성이 뛰어나고, 특히 법률 문서에서 자주 등장하는 합성어나 전문용어를 안정적으로 처리하는 데 강점을 보인다는 점을 검색을 통해 찾아내었다.

이를 확인하기 위해 실제로 본 프로젝트에서는 Komoran, Kkma, Okt, Hannanum 네 가지 형태소 분석기를 같은 문장에 대해 적용하여 비교 실험을 진행하였다. 예제인 "대법원은 이를 공평과 신의칙에 반하는 것으로 판단하였다."라는 문장을 대상으로 분석해본 결과, Komoran은 판단동사인 '판단하였다'를 정확히 '판단하/VV + 었다/EP+EF'로 분리하며 판단 동사의 추출이 용이했다. 반면 Okt는 종종 '판단'과 '하였다'를 분리하지 못하거나 조사와 결합된 상태로 출력해 후처리에 어려움을 주었고, Kkma는 지나치게 세분화하여 판단동사의 탐지가 분산되는 문제점이 있었다.

이처럼 판결문에 특화된 언어 구조를 분석할 때, Komoran은 명확하고 일관된 형태소 태깅 결과를 제공하며, 본 프로젝트의 핵심 작업인 '주어-판단동사-판단내용' 구조 추출의 신뢰성과 효율성을 확보하는 데 가장 적합한 도구로 평가하게 되었다.

2. POS Tagging 및 사건별 고유 동사 출력 (Step 2)

```
# STEP 2 - 품사 태깅 + 동사 추출
print('\n\n')
print('>>>>>>> STEP 2. 품사 태깅으로 고유 동사 목록 추출하기 <<<<<<<')

from konlpy.tag import Komoran

# 1. 형태소 분석기 객체 생성
komoran = Komoran()

# 2. 사건별 동사 추출
all_verbs = []

for case_idx, case_sentences in enumerate(preprocessed_cases, start=1):
    verbs = []
    for sentence in case_sentences:
        tagged = komoran.pos(sentence)

        for word, tag in tagged:
            if tag.startswith('V'):
                verbs.append(word)
        all_verbs.append(verbs)

# 3. 사건별 고유 동사 목록 추출 및 정렬
unique_verbs_by_case = []
for verbs in all_verbs:
    unique_verbs = sorted(set(verbs))
    unique_verbs_by_case.append(unique_verbs)

# 4. 사건별 고유 동사 목록 출력
for idx, verbs in enumerate(unique_verbs_by_case, start=1):
    print(f'\n===== 사건 {idx}의 고유 동사 목록 ({len(verbs)}개) =====')
    print(', '.join(verbs))
```

판결문에서 중요한 문장을 뽑기 위해, 판결문 두 개를 뽑아놓고 읽으며 직접 중요한 문장을 선별해 보았다. 이후 수기로 체크한 결과 서비스의 결과와 비교하기 위한 작업이다. 주어, 동사, 목적어, 필수적 부사어 중에 가장 중요한 것은 '판단된다', '옳지 않다' 처럼 동사라는 결론을 내렸고, 각 사건에 대한 판결에서 중요한 문장을 뽑는 기준을 '동사'로 설정하였다. 이 코드는 앞에서 전처리한 판결문 데이터를 바탕으로, Komoran 형태소 분석기를 사용하여 문장을 품사 단위로 나눈 뒤, 품사 태그 (POS Tagging)가 'VV'(일반 동사)로 시작하는 항목들만 골라내어 동사 목록을 만든다. 이후 사건별로 추출된 동사 목록에서 set을 사용해 중복을 제거하고 sort를 활용해 가나다순으로 정렬한 고유 동사 리스트를 생성하며, 각 사건마다 몇 개의 고유 동사가 사용되었는지 함께 출력한다. 이를 통해 각 판결문에서 사용된 주요 동사들을 확인할 수 있으며, 나중에 쉬운 말 변환 사전을 제작할 때 활용하였다.

3. PMI 기반 Bigram + Trigram Collocation 분석 (Step 3)

```
# STEP 3 - PMI 기반 Bigram + Trigram Collocation 분석
print('\n\n')
print('>>>>>>> STEP 3. 전처리 문장을 기반으로 PMI Bigram/Trigram 분석 <<<<<<<')

# 1. 필요한 라이브러리 불러오기
from konlpy.tag import Komoran
from nltk.collocations import BigramCollocationFinder, TrigramCollocationFinder
from nltk.collocations import BigramAssocMeasures, TrigramAssocMeasures

# 2. 형태소 분석기 객체 생성
komoran = Komoran()

# 3. 전체 문장을 하나의 리스트로 평탄화
all_sentences = []
for case in preprocessed_cases:
    for sentence in case:
        all_sentences.append(sentence)

# 4. 모든 문장을 하나의 문자열로 합치기
joined_text = ' '.join(all_sentences)

# 5. 형태소 분석
words = komoran.morphs(joined_text)

# 6. Bigram Collocation 분석
bigram_finder = BigramCollocationFinder.from_words(words)
bigram_finder.apply_freq_filter(2) # 최소 2회 이상 등장
bigram_measures = BigramAssocMeasures()
top_bigrams = bigram_finder.nbest(bigram_measures.pmi, 5)

# 7. Trigram Collocation 분석
trigram_finder = TrigramCollocationFinder.from_words(words)
trigram_finder.apply_freq_filter(2) # 최소 2회 이상 등장
trigram_measures = TrigramAssocMeasures()
top_trigrams = trigram_finder.nbest(trigram_measures.pmi, 5)

# 8. 결과 출력
print('\n[PMI 기준 상위 2-gram]')
for i, bigram in enumerate(top_bigrams, start=1):
    print(f'{i}. {bigram[0]} + {bigram[1]}')

print('\n[PMI 기준 상위 3-gram]')
for i, trigram in enumerate(top_trigrams, start=1):
    print(f'{i}. {trigram[0]} + {trigram[1]} + {trigram[2]}')
```

그 다음으로는, 판결문에 자주 등장하는 법률 어구와 판단 구조의 반복 패턴을 파악하기 위해 PMI 기반 N-gram 분석을 실시하였다. 분석 대상은 전처리된 모든 문장을 사건 단위로 모은 뒤, 형태소 분석(Komoran)으로 나눈 어절을 기준으로 구성하였으며, N=2인 Bigram과 N=3인 Trigram을 모두 추출하여, 두 단어 혹은 세 단어가 자주 함께 등장하는 조합을 식별하였다. 이 과정에서 수업 시간에 다룬 BigramCollocationFinder와 TrigramCollocationFinder를 사용하였고, 각각 PMI를 기준으로 Collocation 강도가 높은 상위 5개 N-gram을 도출하였다.

처음에는 이러한 N-gram 조합을 기반으로 주요 어구를 자동 조합해 문장을 생성하거나 요약하려는 시도를 했지만, 자연어 처리 측면에서 몇 가지 어려움을 겪었다. 특히 한국어 문법에서 주격 조사(이/가)의 이형태 처리, 보조사의 유동성(은/는, 도, 만, 마저, 조차 등) 등으로 인해, 단어 사이의 관계를 명확하게 규정하기 어려웠고, 실제 문장 생성 결과가 오히려 원문을 천천히 읽는 것보다 이해하기 어렵게 느껴지는 문제가 있었다. 이는 본 프로젝트의 목적이 "법률 비전문가도 이해할 수 있도록 직관적으로 요약"하는 것임을 고려할 때, 방향 전환이 필요하다고 판단하였다.

따라서, 텍스트 생성보다는 의미 중심의 핵심 문장 추출로 방향을 전환하였다. 이를 위해 사건별로 TF-IDF 점수가 가장 높은 문장 1개를 추출하여 주요 문장으로 간주하고, 이후 이 문장을 다시 전처리하여 쉬운 말로 바꾸어 쉬운말 사전을 적용하기로 했다.

4. 사건별 중요 문장 추출 (TF-IDF 기반) (Step 4)

```
# STEP 4 - 사건별 중요 문장 1개씩 추출
print('\n\n')
print('>>>>>>> STEP 4. 사건별 중요 문장 상위 1개 추출 (TF-IDF 기반) <<<<<<<')

from sklearn.feature_extraction.text import TfidfVectorizer
import numpy as np

# 사건별로 중요 문장 추출
case_num = 1
for case in preprocessed_cases:

    if len(case) == 0:
        print(f"###사건 {case_num}: 문장 없음")
        case_num += 1
        continue
    elif len(case) == 1:
        print(f"###사건 {case_num}:")
        print(f"1. {case[0]}")
        case_num += 1
        continue

    vectorizer = TfidfVectorizer()
    X = vectorizer.fit_transform(case)

    sentence_scores = X.sum(axis=1).A1

    top_idx = sentence_scores.argmax()

    # 결과 출력
    print(f"###사건 {case_num}의 중요 문장:")
    print(f"{case[top_idx]}")
    case_num += 1
```

이 단계에서는 사건별로 가장 중요한 문장을 추출하기 위해 TF-IDF 기반 가중치 분석을 활용하였다. 수업 시간에 배운 TfidfVectorizer를 적용하여, 각 사건에서 문장 단위로 벡터화하고, 각 문장마다 TF-IDF 가중치의 합을 계산하였다. 가장 높은 점수를 받은 문장을 해당 사건의 대표 요약 문장으로 간주하여 추출하였다.

5. 불필요한 표현 제거 및 문장 정제 (Step 5)

TF-IDF로 추출된 중요 문장에는 종종 '대법원 선고 판결', '판결요지', '이 사건은' 등 형식적이거나 핵심 정보 이해에 방해가 되는 관용 표현이 포함되어 있었다. 이에 따라 이 단계에서는 re.sub()를 이용해 미리 정의한 표현들을 제거함으로써 문장을 간결하고 핵심 중심으로 정제하였다. (데이터 재처리) 이는 수업 시간에 배운 불용어 제거 및 전처리 함수 구성 방식을 그대로 응용한 것으로, 중요 문장을 보다 직관적이고 간결하게 전달할 수 있도록 하였다. 이 과정을 통해 법률적 문장 구조를 해치지 않으면서도 문장의 의미 전달력을 높일 수 있었다.

```
important_sentences = []

for case_num, case in enumerate(preprocessed_cases, start=1):
    print(f"###사건 {case_num}")

    if len(case) == 0:
        print("문장이 없어 처리하지 않음.")
        important_sentences.append("")
        continue

    # TF-IDF 기반 가장 중요한 문장 1개 선택
    if len(case) == 1:
        important = case[0]
    else:
        vectorizer = TfidfVectorizer()
        X = vectorizer.fit_transform(case)
        sentence_scores = X.sum(axis=1).A1
        top_idx = sentence_scores.argmax()
        important = case[top_idx]

    # 불필요한 표현 제거
    for phrase in remove_phrases:
        important = re.sub(phrase, '', important)

    important_sentences.append(important)
    print(f"중요 문장 (정제 후): {important}")

# STEP 5 - 중요 문장 추출 및 불필요한 표현 제거
print('\n\n')
print('===== STEP 5. 사건별 중요 문장 추출 및 불필요한 표현 제거 =====')

import re
from sklearn.feature_extraction.text import TfidfVectorizer
import numpy as np

# 제거할 관용 표현
remove_phrases = [
    r'대법원\s*선고\s*판결',
    r'판결\s*요지',
    r'이 사건은',
    r'결론',
    r'주문과\s*같이\s*판결한다'
]
```

6. 쉬운 말 해설 추가 (용어 설명 사전 적용) (Step 6)

판결문은 법률 용어와 특수한 문장 구조로 인해 일반 대중이 이해하기 어려운 경우가 많다. 이를 개선하기 위해, 이 단계에서는 수작업으로 구성한 easy_dict 사전을 기반으로, 중요한 법률 용어나 동사(예: '인용하', '파기하', '채무', '청구' 등)에 대해 쉬운 말로 의미를 부연 설명하였다. 구체적으로는 re.sub()를 이용하여 해당 단어가 등장할 때마다 괄호를 붙여 설명을 자동 삽입하는 방식을 사용하였다. 예컨대 '피고'는 '피고(소송을 당한 사람)'으로, '청구'는 '청구(요청)'으로 변환되어 출력된다. 이는 수업 시간에 배운 정규표현식 기반 전처리와 단어 사전 기반 반복 처리를 응용한 것이며, 결과적으로 법률 비전문가도 핵심 내용을 직관적으로 파악할 수 있도록 도와주는 해설 기능으로 작동한다.

쉬운말 변환 사전을 실행하기 위해 찾아보니, 각 단어간의 의미 거리를 수치화하고, 이를 바탕으로 변경하는 절차가 필요하다는 것을 알게 되었다. 이에, 해당 내용은 불가능하다고 생각해 수동으로 직접 쉬운말 변환 사전을 제작했다. 쉬운말 변환 사전은 '표준국어대사전'을 찾아가며 작성하였고, 모든 결과를 다 포함하지 않을 수 있다. 아울러, 판결문의 경우 수기로 작성되며 맞춤법에 대한 검토는 진행하나, 한글맞춤법 규정을 모두 완벽하게 지켜야만 하는 문서는 아니기에, 일부 띄어쓰기 등으로 인해 잘못 인식되었을 가능성도 배제할 수 없다. 위에 언급했다시피, 각 단어간의 의미 거리를 수치화하고 이를 바탕으로 가장 가까운 단어를 찾는 서비스를 도입한다면 서비스의 원 목적인 '판결문을 쉬운 말로 변환하여 일반 국민에게 제공하겠다'는 목표에 더 부합하는 서비스가 될 것이라 생각한다.

다음은 해당 프로젝트 속에서 진행한 쉬운말 변환 사전의 내용이다. 판결문 일부를 읽으며 선별한 동사와, 홍보대사 활동을 하며 체감한 개인적 기준에 의거해 피고, 원고와 같은 일부 명사도 수정해두었다.

※ 변환 사전 내용 : 표준국어대사전을 활용해 작성

```
easy_dict = {  
    # 판단 관련 동사  
    "기각하": "요청을 받아들이지 않음",  
    "인용하": "요청을 받아들임",  
    "간주하": "그렇게 본다",  
  
    # 판결 처리  
    "파기하": "이전 판결을 무효로 함",  
    "환송하": "다시 판단하도록 보냄",  
    "취소하": "효력을 없앴",  
  
    # 수용·부정 계열  
    "수용하": "받아들임",  
  
    # 법적 의무와 책임  
    "위반하": "규칙을 어김",  
    "면제하": "책임을 면함",  
    "부담하": "책임이나 의무를 가짐",  
  
    # 법적 효과  
    "성립하": "성립되었다고 봄",  
    "효력있": "법적인 효력이 있음",  
    "효력없": "법적인 효력이 없음",  
    "위법하": "법을 위반함",  
    "적법하": "법에 맞음",  
    "정당하": "옳고 타당함",  
    "정당화되": "정당하다고 인정됨",  
    "반하": "반함",  
    "미치": "영향이 미침",  
  
    # 확신·의심  
    "추정하": "그렇다고 예상함",  
    "추단하": "추론하여 판단함",  
  
    # 일반 명사  
    "원고": "소송을 건 사람",  
    "피고": "소송을 당한 사람",  
    "판결": "재판 결과",  
    "청구": "요청",  
    "이행": "실제로 함",  
    "계약": "약속",  
    "채무": "갚아야 할 돈 또는 의무",  
    "채권자": "돈을 받을 사람",  
    "채무자": "돈을 갚아야 하는 사람",  
    "의무": "해야 할 일",  
}
```

VI. 최종 결과 출력물



>>>>>> STEP 1. 데이터 전처리 (문장 단위 토큰화 + 기호 위주 불용어 제거) <<<<<<<

=== 전처리된 결과 ===

===== 사건 1 =====

1. 대법원 선고 판결 소유권이전등기 아파트 매매계약서에 인도일과 실제 명도일 약정이 별도로 있는 경우 매도인의 현실인도의무 인정 여부가 문제된 사건 판결요지 일반적으로 계약
2. 계약 내용이 명확하지 않은 경우 계약서의 문언이 계약 해석의 출발점이지만 당사자들 사이에 계약서의 문언과 다른 내용으로 의사가 합치된 경우 의사에 따라 계약이 성립한 것
3. 당사자 사이에 계약의 해석을 둘러싸고 이견이 있어 당사자의 의사 해석이 문제 되는 경우에는 계약의 형식과 내용 계약이 체결된 동기와 경위 계약으로 달성하려는 목적 당사자의
4. 민법 행에서 정한 선이행의무를 지고 있는 당사자가 상대방의 이행이 곤란할 현저한 사유가 있는 때에 자기의 채무이행을 거절할 있는 경우 선이행채무를 지고 있는 당사자가 계약
5. 상대방의 채무가 아직 이행기에 이르지 않았지만 이행기에 이행할 것인지 여부가 현재로 불확실하게 경우에는 선이행채무를 지고 있는 당사자라도 상대방의 이행이 확실하게 때까;
6. 갑이 을로부터 아파트를 매수하기로 하는 계약을 체결하였고 계약 체결 무렵 아파트에 거주 중인 임차인 병이 임대차계약기간 만료 후 계약갱신요구권을 행사하지 않고 아파트를 ;
7. 내용과 특약사항으로 실제명도일을 정하고 있고 매매계약의 문언 해석상 쌍방이 을의 현실인도의무 이행일은 실제명도일로 하되 임차인 병에 대한 아파트의 반환청구권 양도에 의
8. 원심의 판단 원심은 사건 매매계약의 해석상 피고가 까지 사건 아파트를 원고에게 현실인도할 의무가 인정되지 않으므로 의무 불이행의 영리가 있음을 이유로 한 원고의 진금 지급
9. 법원의 판단 그러나 원심의 이러한 판단은 다음과 같은 이유로 수긍하기 어렵다
10. 일반적으로 계약을 해석할 때에는 형식적인 문구에만 얽매어서는 되고 쌍방당사자의 진정한 의사가 무엇인가를 탐구하여야 한다
11. 계약 내용이 명확하지 않은 경우 계약서의 문언이 계약 해석의 출발점이지만 당사자들 사이에 계약서의 문언과 다른 내용으로 의사가 합치된 경우 의사에 따라 계약이 성립한 것;
12. 당사자 사이에 계약의 해석을 둘러싸고 이견이 있어 당사자의 의사 해석이 문제 되는 경우에는 계약의 형식과 내용 계약이 체결된 동기와 경위 계약으로 달성하려는 목적 당사자;
13. 정하고 있고 특약사항으로 실제명도는 한다
14. 정하고 있다

>>>>>> STEP 2. 품사 태깅으로 고유 동사 목록 추출하기 <<<<<<<

===== 사건 1의 고유 동사 목록 (21개) =====

가리키, 관하, 까, 나아가, 대하, 되, 둘러싸, 따르, 말미암, 미치, 반하, 발, 보, 생기, 서, 얹매이, 의하, 이르, 있, 정하, 하

===== 사건 2의 고유 동사 목록 (13개) =====

가지, 관하, 나타내, 대하, 따르, 면하, 미치, 의하, 인하, 있, 정하, 준하, 하

===== 사건 3의 고유 동사 목록 (21개) =====

가지, 관하, 구하, 까, 다니, 대하, 들, 따르, 미치, 바귀, 발, 밝히, 보, 보이, 비추, 살, 살펴보, 위하, 있, 찾, 하

===== 사건 4의 고유 동사 목록 (24개) =====

가하, 갈치, 관하, 까, 넘, 누리, 대하, 되, 두, 따르, 미치, 받아들이, 보, 보이, 비추, 살펴보, 삼, 위하, 의하, 인하, 있, 정하, 줄어들, 하

>>>>>> STEP 3. 전처리 문장을 기반으로 PMI Bigram/Trigram 분석 <<<<<<<

[PMI 기준 상위 2-gram]

1. 담보 + 채무액
2. 덕행 + 명성
3. 맺 + 을지
4. 선비 + 인센티브
5. 어느 + 하나에

[PMI 기준 상위 3-gram]

1. 연구 + 인력 + 개발비
2. 품성 + 덕행 + 명성
3. 피 + 담보 + 채무액
4. 재 + 심판 + 정
5. 덕행 + 명성 + 신용

>>>>>> STEP 4. 사건별 중요 문장 상위 1개 추출 (TF-IDF 기반) <<<<<<<

사건 1의 중요 문장:

내용과 특약사항으로 실제명도일을 정하고 있고 매매계약의 문언 해석상 쌍방이 을의 현실인도의무 이행일은 실제명도일로

사건 2의 중요 문장:

원심의 이유 실시에는 일부 적절하지 않은 부분이 있으나 원고가 피고 양우건설에 대하여 가지는 하자보수를 갈음하는 손

사건 3의 중요 문장:

이는 임대인의 주거 상황 임대인이나 그의 가족의 직장이나 학교 사회적 환경 임대인이 실제 거주하려는 의사를 가지게 경

사건 4의 중요 문장:

원심의 판단 원심은 취업규칙 근로계약서상의 시업시각과 업무일지상의 업무종료시각 사이의 시간 휴게시간 시간을 제외한

사건 5의 중요 문장:

상고이유에 관하여 저작권법이 보호하는 것은 인간의 사상 또는 감정을 문자 등에 의하여 구체적으로 외부에 표현하는 창

🔗 ===== STEP 6. 쉬운 말 해설 (용어 설명 사전 적용) =====

[사건별 중요 문장 요약 (용어 설명 포함)]

사건 1의 중요 문장 요약:

1) 내용과 특약사항으로 실제명도일을 정하고 있고 매매계약의 문언 해석상 쌍방이 을의 현실인도의무 이행일은 실제명도일로 하되

사건 2의 중요 문장 요약:

1) 원심의 이유 설시에는 일부 적절하지 않은 부분이 있으나 원고가 피고(소송을 당한 사람) 양우건설에 대하여 가지는 하자보수청구

사건 3의 중요 문장 요약:

1) 이는 임대인의 주거 상황 임대인이나 그의 가족의 직장이나 학교 사회적 환경 임대인이 실제 거주하려는 의사를 가지게 경위 {

사건 4의 중요 문장 요약:

1) 원심의 판단 원심은 취업규칙 근로계약서상의 시업시각과 업무일지상의 업무종료시각 사이의 시간 휴게시간 시간을 제외한 나머지

===== STEP 5. 사건별 중요 문장 추출 및 불필요한 표현 제거 =====

[사건 1]

중요 문장 (정제 후): 내용과 특약사항으로 실제명도일을 정하고 있고 매매계약의 문언 해석상 쌍방이 을의 현실인도의무 이행일은 실제명도일로 하되

[사건 2]

중요 문장 (정제 후): 원심의 이유 설시에는 일부 적절하지 않은 부분이 있으나 원고가 피고 양우건설에 대하여 가지는 하자보수를 갈음하는 손해배상청

[사건 3]

중요 문장 (정제 후): 이는 임대인의 주거 상황 임대인이나 그의 가족의 직장이나 학교 사회적 환경 임대인이 실제 거주하려는 의사를 가지게 경위 임대

[사건 4]

중요 문장 (정제 후): 원심의 판단 원심은 취업규칙 근로계약서상의 시업시각과 업무일지상의 업무종료시각 사이의 시간 휴게시간 시간을 제외한 나머지

[사건 5]

중요 문장 (정제 후): 상고이유에 관하여 저작권법이 보호하는 것은 인간의 사상 또는 감정을 문자 등에 의하여 구체적으로 외부에 표현하는 창작적인

[사건 6]

중요 문장 (정제 후): 주식회사가 지방세법 개정 전에 지출한 조세특례제한법 법률 호로 개정되기 전의 조의 항 조세특례제한법 법률 호로 개정되기 전

[사건 7]

VII. 결론 및 한계

1. 초기 목표와 달라진 부분

프로젝트의 초기 목표는 대법원 판결문으로부터 ‘행위 주체(예: 원고·피고)’, ‘행위(예: 청구, 위반, 인용)’, ‘결론(예: 파기환송, 무죄)’ 등 핵심 판단 요소를 자동으로 구조화하고, 일반인이 직관적으로 이해할 수 있도록 요약 및 시각화 기능을 제공하는 것이었다. 그러나 실제 판결문 데이터를 다뤄보며, 문장의 복잡성과 용어의 다양성, 판결 형식의 불균일성 등으로 인해 당초 의도했던 정형 데이터 수준의 자동 추출이나 명확한 결론 분류가 어렵다는 한계를 마주하게 되었다. 특히 주어와 결론 사이의 의미상 거리, 피동 표현, 법적 관용구 등이 모델 없이 단순 규칙 기반으로는 처리하기 어려웠으며, 구조적 요약보다는 중요 문장 추출과 쉬운말 해설 제공이라는 현실적인 방향으로 프로젝트 범위를 조정하게 되었다. 이는 수업 시간에 배운 기술들을 가장 안정적으로 적용할 수 있는 영역에 집중한 결과로, 실제 사용자에게 핵심 문장을 전달하고 해석을 도와주는 서비스 구성으로 방향을 전환하여 진행하였다.

2. 서비스 성과 요약

제공한 서비스는 서브 시스템 2개, 메인 시스템 1개이다. 서브 시스템은 데이터 크롤링만으로 얻어낸 원본 데이터 (raw-data)를 활용해 출력할 수 있는 시스템이고, 메인 시스템은 데이터 크롤링, 전처리, 언어학적 분석 모두를 거쳐야 출력할 수 있는 시스템이다.

서브 시스템 1 : 크롤링한 판결문 (raw-data)을 넣으면 ‘결과’를 기준으로 판결문들을 분류해준다. 결과는 A유형 (상고 기각), B유형 (원심판결 파기)로 나뉘며, 하단에 상고 기각과 원심판결 파기에 대한 설명이 함께 출력된다.

서브 시스템 2 : 크롤링한 판결문 (raw-data)을 넣으면 판결의 제목을 모아서 출력한다.

메인 시스템 : 이번 프로젝트의 메인 시스템은 법률 문서인 판결문을 대상으로, 수업 시간에 배운 자연어처리 기법을 충실히 이행하여 전처리부터 핵심 문장 추출, 의미 해설 제공까지의 전체 텍스트 처리 흐름을 구축하는 것이었다. 텍스트는 대법원 판결문 형식에 맞춰 수집되었으며, 각 사건은 <대법원 ... 판결>과 같은 제목 단위로 분할한 뒤, 본문과 함께 사건별 단위 리스트로 정리하였다. 익숙하지 않은 사용자도 판결문 내 핵심 의미를 파악할 수 있도록 돕는 결과를 얻었다.

텍스트 처리에 사용된 언어처리 도구로는 KoNLPy의 Komoran 형태소 분석기, NLTK의 N-gram 연어 추출 도구, Scikit-learn의 TfidfVectorizer가 있으며, 수업 시간에 실습한 내용과 도구를 기반으로 구성하였다. 전처리 단계에서는 수업 시간에 다룬 정규표현식인 re.sub를 활용하여 한 글자 불용어, 특수 기호, 필요 없는 기호를 제거하였고, 종결 어미 표현인 '다.', '요.'를 기준으로 문장을 문장 단위로 분할하였다. 이후 형태소 분석기는 Komoran을 사용하여 각 문장의 동사를 추출하고, 사건별 고유 동사 목록을 정렬 및 출력하였다.

이후 NLTK 기반의 BigramCollocationFinder, TrigramCollocationFinder와 PMI 연관도 지표를 활용하여, 전체 판결문 텍스트 내에서 반복적으로 사용되는 2-gram, 3-gram 표현을 추출하였다. 이를 통해 사건 전반에서 자주 나타나는 법률적 문맥 패턴을 탐색할 수 있었다. 핵심 문장 추출 단계에서는 TfidfVectorizer를 활용하여, 각 사건 내 문장별 TF-IDF 가중치의 합을 계산하고, 가장 높은 점수를 받은 문장을 중요 문장으로 선택하였다.

추출된 중요 문장은 다시 정규표현식을 활용해 '대법원 선고 판결', '판결요지' 등의 형식적 문구를 제거하였고, 쉬운 언어로 바꾸기 위해 수작업으로 구성한 법률용어 해설 사전(easy_dict)을 적용하였다. 이 과정 역시 수업에서 학습한 re.sub()기반 반복 처리 방식을 그대로 응용한 것으로, '피고' → '피고 (소송을 당한 사람)'과 같이 일반인이 이해하기 쉬운 방식으로 문장을 변환하였다. 이를 통해 시스템 제작 결과, 각 판결문에서 추출된 핵심 문장은 일부 문장은 다소 어색하였지만, 어느 정도 실제 사건의 핵심 논지를 반영하고 있었고, easy_dict적용 이후의 문장은 원문의 구조를 해치지 않으면서도 의미 해설을 적절히 추가하여 정보 접근성을 높이겠다는 목적을 일부 달성한 것으로 보였다.

3. 한계점

데이터 수집 단계 : 이번 프로젝트에서는 대법원 사법정보공개포털의 판례 섹션이 자동 크롤링이 불가능했고, BeautifulSoup을 통한 정적 크롤링과 Selenium을 통한 동적 크롤링 모두 불가능했다. 추가로, 판결 요지 부분의 버튼을 누르면 팝업창이 연결되는데, 팝업창 크롤링 또한 불가능했다.

해당 방법 이외에도 대법원 판결 열람 사이트에서 자동 크롤링이 가능해진다면 해당 프로그램을 확장하여 통계적 분석으로도 활용할 수 있을 것이다. 만일 이를 학습 가능한 모델로 만든다면 더 많은 판결문 언어 데이터를 학습해 본 프로젝트의 원 목적이었던 '판단 주체, 판단 대상, 판단 결과'의 구분과 '쉬운 말로의 변환'의 결과가 인간이 하나하나 작성한 것처럼 매끄러워질 수 있을 것으로 예상된다.

프로젝트 목적 달성의 어려움 : 본 프로젝트의 목적은 대법원 판결문을 크롤링하여 서비스 대상자에게 '판결문의 핵심 요지'를 몇 문장으로 간추려 제공하고, '판단 주체, 판단 대상, 판단 결과'를 구분해 법적 관계를 명확히 정리해주고, '쉬운 말로의 변환'을 통해 법 언어를 충분히 알지 못하는

일반 국민들도 판결문을 읽고 이해할 수 있도록 하는 것이었다.

데이터 수집 이후로도 해당 프로젝트를 진행하며 모든 단계에서 어려움에 부딪혔는데, 판결문의 핵심 요지를 몇 문장으로 간추리는 게 어려웠다. 직접 판결문을 뽑아 중요한 부분을 형광펜으로 칠해야 하는 경우, 직관과 보유한 지식에 기반해 선별할 수 있었으나, 암묵적으로 사용하는 지식을 코드로 구현해 프로그램이 동일한 방식으로 처리해 같은 결과를 출력하도록 하는 것이 어려웠다. 실제로 골라진 문장이 완벽하게 골라졌다고는 할 수 없으나, 그래도 많은 텍스트에서 그나마 관련도가 높은 3-5문장을 골라내고 있으며, 판결 요지에 그래도 포함된다고 할 수 있는 문장들이 선별되었다. 아마 해당 부분을 기계 학습을 시킬 수 있는 모델을 설계하여 판결문의 구조를 학습시키고 판결문의 핵심 요지가 어디인지 정답과 함께 학습시켜 핵심 요지를 추출하게 한다면 매우 활용도 높은 모델이 될 것이라 생각한다.

판단 주체, 판단 대상, 판단 결과를 구분해 법적 관계를 명확히 하는 것도 어려웠는데, 예상으로는 문장 단위로 토큰화를 하고 나면, 한 문장 안에 주어, 동사가 하나씩, 그에 포함되는 주요 성분이 개수에 맞게 존재하여 하나의 매끄러운 문장을 새로 구성해낼 수 있을 줄 알았는데, '-가' 형태는 모두 주어로 처리하나 '-은' 형태는 주어로 인식하지 않은 경우도 많았고, 한국어 문장을 러프하게 '-다', '-요'로 토큰화하여 변수에 저장해두니 안은-안긴 문장 혹은 이어진 문장 등 겹문장에 대해 제대로 인식하기가 어려웠다. 또한, 한국어에 존재하는 이형태 ('이', '가')에 대해 받침의 유무를 기준으로 다르게 설정하는 것이 어려웠고, 한국어에 존재하는 '보조사' ('은', '는', '도', '만', '마저', '조차', '까지')는 특정한 격을 가지지 않은 채로 격 조사를 대신하는데, 그래서 주어가 아닌데도 주어처럼 인식하게 되어 정상적인 형태의 문장을 생성해내기 어려웠다.; 결국 토큰화를 통해 한 문장으로 설정해 놓은 문장에서 주어 3개, 동사 2개, 목적어 1개 이런 식으로 도출되어 새로운 문장으로 구성할 수 없었다. 이에, 전처리 단계에서 겹문장에 대해 모델을 학습시켜 전처리하거나, 전처리를 해당 문장의 중심 주어, 동사 등의 주요 문장 구성 성분만 뽑아내어 Dependency Parsing을 진행한다면 보다 수월하게 문장 생성이 가능할 것 같다.

쉬운말 변환에서도 단어 간의 의미 상 거리를 수치화하여 자동적으로 단어에 해당하는 의미를 고려해 변환하는 것이 불가능하여, 일부 판례와 출력 결과를 활용해 수동으로 쉬운말 변환 사전을 제작하였다. 추후 단어 간의 의미를 매핑하고, 광범위한 언어 Corpus를 학습해 단어 사용의 맥락까지 학습한 다음, 판결문 용어와 표준국어대사전의 의미를 함께 학습시켜 맥락에 맞는 법적 단어를 쉬운 형태로 변환하는 프로그램을 만들면 좋을 것 같다.

메인 시스템 : 이번 프로젝트는 수업 시간에 배운 자연어처리 기법을 충실히 적용하여 실제 판결문 데이터를 처리하고 해석 가능한 문장으로 재가공한 유의미한 시도였지만, 해결해야 할 점도 많이 남아 있는 성과였다.

우선, KoNLPy의 Komoran 분석기는 상대적으로 빠르고 간결한 분석을 제공하지만, 전문 법률 용어나 긴 복합 명사, 접속어 중심 구조를 가진 판결문에서는 의미 단위의 분절이 부정확하게 처리되는 경우가 있었다. 예를 들어, "책임을 부담한다"와 같은 표현이 "책임", "을", "부담하"로 나뉘거나, "소유권 이전등기" 같은 복합 명사가 의미 있는 단위로 분리되지 않는 문제가 발생했다.

다음으로 진행한 PMI 기반 N-gram 분석에서 의미상 중요도가 아닌 우연에 의한 단순 빈도상으로

데이터가 추출되는 경우가 존재했다. PMI는 통계적으로 강력한 Collocation 추출 지표이지만, 텍스트 양이 적거나 문장이 짧은 사건에서는 충분한 빈도와 어떤 조건 하에서 공동으로 등장하는지에 대한 정보가 확보되지 않아 의미 있는 Collocation을 포착하지 못하거나, 우연적 결합이 높은 PMI 값을 갖는 경우가 있었다. 즉, 텍스트 규모에 따라 N-gram 결과의 신뢰도가 편차를 보일 수 있음을 확인했다. 또한, 법률 문장 특유의 복잡한 구조로 인해 TF-IDF 기반 중요 문장 추출이 의미의 중심이 아닌 형식적 반복 단어에 의해 영향을 받을 가능성 또한 존재한다. 판결문은 종종 "이 사건", "이 법원은", "판시한 바와 같이"와 같은 표현을 자주 사용하기 때문에, TF-IDF 벡터의 가중치가 핵심 법리보다 주변 서술에 치우칠 위험도 있었다.

쉬운말 사전(easy_dict)의 수동 작업으로 인해, 단어 선정 과정에서 주관적 기준이 개입하였다는 아쉬움이 남는다. 해당 사전은 직접 단어를 선정하고 설명을 붙이는 방식으로 구성되었는데, 이 과정에서 불가피한 주관적 요소의 개입으로 어휘 선택의 일관성이 흔들릴 여지가 있었다.

마지막으로, 전체 프로젝트는 형식적 구조가 정제된 판결문 데이터를 대상으로 제한된 수의 사건에 적용되었기 때문에, 실제 법률 서비스에 적용하기 위해서는 다양한 판결 유형, 양식, 문장 길이, 주제 분야에 대한 일반화 가능성을 확인할 후속 작업이 필요하다고 생각한다.

4. 개선 방안, 향후 연구 활용 방안

서브 시스템 1 : 현재는 해당 결과를 단순히 확인하는 것 정도에 그쳤지만, 추후 대법원 판결 열람 사이트의 자동 크롤링이 가능하게 되면 대법원 판결에 대한 통계를 계산하는 것이 가능해질 것으로 예상된다. 원심파기와 상고 기각 중에 어떤 결론이 더 우세한지를 확인할 수도 있으며, 대법원 판결문 최하단에는 해당 판결에 참여한 법관의 이름이 나열되는데, 이를 활용하면 추가로 어떤 법관이 판결에 참여했을 때 혹은 어떤 법관의 조합일 때 A 혹은 B유형이 자주 결론이 내려지는지를 확인할 수 있을 것이다. 물론, 해당 법관이 들어간 재판마다 모두 타당한 절차와 심사를 거쳐 한 쪽에 치우친 결과가 나왔을 가능성을 배제할 수는 없다. 다만, 대법원의 최종 판결에 대한 신뢰도를 높이기 위해, 법관들에게 해당 내용을 제시하고, 재판 이후에 주기적으로 재판 절차 및 사고의 타당성을 짚어보는 사후 절차가 이러한 시스템을 통해 간단히 진행될 수 있다면, 법관 스스로에 대한 자정 작용도 기대해 볼 수 있을 것이다.

서브 시스템 2 : 해당 부분은 단순히 제목만을 추출해 내었지만, 추후 활용 가능성이 매우 큰 서비스이다. 실제 공익과법을 연계전공으로 수학하며 형법과 상법 시험을 준비했었는데, 특정 상황에 대한 대법원 판례를 숙지하고 있는 것이 중요했다. 대개 변호사 시험 사례형 문항을 기반으로 시험 문제가 출제되기에, 특정 상황이 주어지는 경우에 있어 법리적 판단을 하는 것이 필요한데, 학설이 대립하는 경우 대법원의 판결을 답으로 적어야 한다. 대법원의 판결은 하급심 법 기관과 학설 사이의 대립을 해소하여 하나의 결론이 나게끔 해주는 역할을 하기 때문이다. 또한, 변호사 실무에서도 관련 판례를 빠르게 찾아보는 것이 매우 중요한데, 해당 부분 코드를 약간 수정하여 전체 텍스트 중 '특정 키워드가 포함된 경우'라는 조건문에 해당하는 코드를 추가한다면 해당 조건에 맞는 판례만도 따로 뽑아낼 수 있을 것으로 보인다. 현재 아직 학부생이기에 직접 경험해보진 못하였으나, 대법원 홍보대사로 대법원 방문하였을 때, 내부적으로 판례 검색 시스템을 활용하고 있다는 얘기를 들은 적 있다. 보안 상의 이유와 실무상 이점으로 인해 내부 시스템을 국민에게 공개하는 것은 어렵겠지만, 법은 일반 국민의 삶 전반을 규율하는 제도이므로 해당 서비스와 같이 간략하게 검색할 수 있는 기능을 추가해준다면, 일반 국민이 상소 전에 판례를 검색해보는 행위를 통해 신체적, 심리적 부담과 시간적, 재정적 부담이 큰 재판을 이어가도 승산이 있을지, 없을지를 미리 확인하여 보다 합리적인 판단에 기여할 수 있을 것으

로 보인다.

메인 시스템 : 우선, 전체적으로 문장이 길기 때문에 의미상으로 문장을 더 추가로 나누어 분석하는 것이 좋을 것으로 생각된다. 겹문장 중 안은-안긴 문장 등을 더 세심히 분석해 낼 수 있는 언어학적 분석 방식을 사용한다면 보다 나은 결과가 도출될 것으로 보인다. 또한, N-gram 추출의 신뢰도를 높이기 위해 단순 빈도 기반 PMI 외에도 TF-IDF 가중치나 문맥 기반 연관도를 적용하거나, 사건을 그 성질에 따라 분류하고, 추가적인 조건을 달아 필터링을 더 거치면 보다 의미있는 Collocation 및 N-gram을 얻어낼 수 있을 것 같다. 이를 모아서 문서 형태로 배포한다면 특히 외국인이 한국의 법 체계를 학습할 때 유용하게 사용될 것 같다. 실제로 공익과 법 연계 전공에는 유학생과 교환학생이 많은데, 이들에게 배포한다면 학습에 큰 도움이 될 것이다. 중요 문장 추출의 경우에도 단어 빈도 기반이 아닌 키워드 필터링 또는 Dependency Parsing을 기반하여 핵심어를 찾는 방식으로 정말 중요성이 높은 문장을 뽑아내는 방식으로 보완한다면 서비스의 원 목적을 더 충실히 달성할 수 있을 것으로 보인다. 쉬운말 변환 사전인 easy_dict 역시 우선 단어의 양을 대폭 늘리고, 사용자 피드백 기반으로 보완하거나 전문가 감수를 거치고, 나무위키처럼 사용자가 입력하여 그 데이터베이스를 확장하는 방식을 활용하는 방법으로 많은 경우를 추가하는 것이 방대한 언어데이터를 다루는 데 있어 좋을 것 같다는 생각이 든다. 마지막으로, 자동 크롤링 방식을 추가로 도입해 다양한 유형의 판결문 데이터를 확보하고, 모델의 성능을 평가할 객관적인 성능 평가 지표를 추가한다면 실제로 활용 가능한 수준으로도 발전시킬 수 있을 것이라 생각한다.

부록

프로젝트를 마치며

해당 프로젝트를 통해 살펴본 판결문의 문장부호와 종결어미로 나뉜 문장 한 개는 보편적인 문장 한 개에 비해 더 많은 내용을 담고 있었다. 마침표가 찍힐 때까지 실질적으로 문장이 3개가 넘어가는 내용을 포함하는 문장도 존재했다. 이는 초창기 판결문이 일반인이 이해하기 어려운 정도로 복잡하고 어렵게 쓰여있었고, 현재 이를 해소하기 위해 응용언어학 등의 분야에서 법 언어학에 대해 논하고, 실제 실무에서도 쉬운 판결을 쓰자는 자정작용이 존재함에도 불구하고 여전히 판결문이 신문 기사, 인터뷰 등의 보편적 텍스트에 비해 어려운 구조를 띄고 있음을 시사한다고 생각한다.

처음 시도했던 Dependency Parsing이 온전히 되지 않았고, 주체와 객체가 여러 개 출력되었던 부분에서는 판결문의 마침표 기준 상 한 단위의 문장에서 주체와 객체의 관계가 명시적으로 드러나 있기 보다는 세부적으로 확인하여야 그 관계를 명확히 알 수 있을 정도로 복잡하게 쓰여있음을 보여준다. 이는 단순히 판결문이 어렵게 쓰여졌다고만 보기는 어렵고, 법의 기본적 성질에 의한다고 분석했다. 법은 문제해결의 수단인데, 기본적으로 현대 사회에서 '대화'를 통해 갈등을 해결하는 것이 가장 바람직하다. 다만, 원만한 합의에 이르지 못하였을 경우, 최후의 수단으로 법을 찾게 되는데, 이 경우 갈등 관계는 매우 첨예하고 각 측의 입장에서 모두 타당한 측면이 존재한다. 아울러, 법은 문언적 해석과 법의 입법 취지를 고려한 문언 너머의 해석 모두가 반영되어 판결이 결정되기에 이미묘하고 복잡한 지점을 기록으로 남기고 타 갈등의 모범 사례로 활용하는 대법원 판결문은 그 복잡성이 높을 수 밖에 없을 것이다. 앞으로의 법 언어 분석과 이에 프로그래밍 언어를 도입하는 경우에, 판결문의 복잡성을 충분히 고려하여 연구를 진행해야 한다는 생각이 들었다.

프로젝트를 마치며 들었던 생각은, 법조계 직무를 희망하는 사람이자, 대법원 홍보대사로 활동하며 국민의 법 감정과 배치되는 판결이 나오는 경우 그에 대해 크게 분개하는 시민들의 모습을 가까이서 느꼈다. 법을 배우기 전에는 나 또한 그러한 흐름의 일부였지만, 헌법과 형법을 배운 뒤로는 입법 취지와 그러한 법조문이 존재해야만 했던 이유를 이해하게 되어 법조인과 일반 국민간의 입장 차이를 좁히고 싶다는 생각이 늘 들어왔다. 대법원 유튜브 개편을 진행하던 중, 프로젝트를 시작하게 되었고 프로젝트의 시작도 일반 국민이 법을 조금 더 알게 된다면 시각 차가 줄어들지 않을까 하는 생각에서였다. 분개하는 목소리 뒤로 항상 나오는 말이 있다. "AI 판사로 다 대체해야 한다."는 말이다. 이 프로젝트를 마치고 나니, 판결문이라는 자연어의 복잡함에 대해 깊이 체감하게 되었다. 문언과 문언 너머의 해석 사이의 타협점을 찾고, 세상에 존재하지 않았던 갈등 양상을 분석하여 판결하는 등 기계 학습으로도 대처하기 어려운 내용들이 있어 해당 말에 대한 걱정이 매우 줄어들었다. 물론, 기계 학습을 통해 더 많은 판례들을 더 짧은 시간에 검토하여 타당한 판결을 내릴 수 있게 되테고, 판결의 주체가 사람에서 기계로 넘어갈 순 있겠지만, 결국 그 과정을 검토하고 제대로 법리가 구현되고 있는지는 앞으로도 사람이 담당할 부분으로 남을 것 같다는 생각을 했다. 기계와 통계는 유용하나 그 변수에 대해 늘 들어맞는 것은 아니라고 생각했다. 인간 또한 완전무결한 존재는 아니지만, 그로 인해 문언과 통계 이면의 무언가를 찾아낼 수 있는 존재라고 생각한다.

프로젝트 전반에서 몇 만 글자나 되는 텍스트를 단 몇 초 안에 분석하여 일련의 결과를 내는 기계에 대한 전망의 순간과 더불어, 인간만이 할 수 있는 행위에 대해 깊게 체감하게 된 시간이어서 많은 것을 배우고 느끼게 된 순간이었다.